

Spectra AI: Agentic Penetration Testing

By: Ritvik Indupuri

Date: 8/22/2025

1. Introduction

Spectra AI is an advanced penetration testing framework that leverages a multi-agent AI system to automate and streamline security assessments. It combines the strategic, cognitive capabilities of modern Large Language Models (LLMs) with the practical execution of industry-standard cybersecurity tools. The core philosophy is to create an intelligent system that mimics a human penetration tester's workflow: gathering intelligence, devising a strategy, executing tool-based scans, analyzing the output, and generating actionable reports.

2. System Architecture

The application is a full-stack Next.js application, featuring a React-based frontend and a Node.js backend. This architecture enables seamless integration between the user interface and the server-side AI logic.

2.1. Architectural Diagram

The following sequence diagram illustrates the detailed data flow and interaction between the specialized AI agents and services when a user initiates a test. It shows the complete process, from the initial reconnaissance phase to the final report aggregation by the coordinator agent.

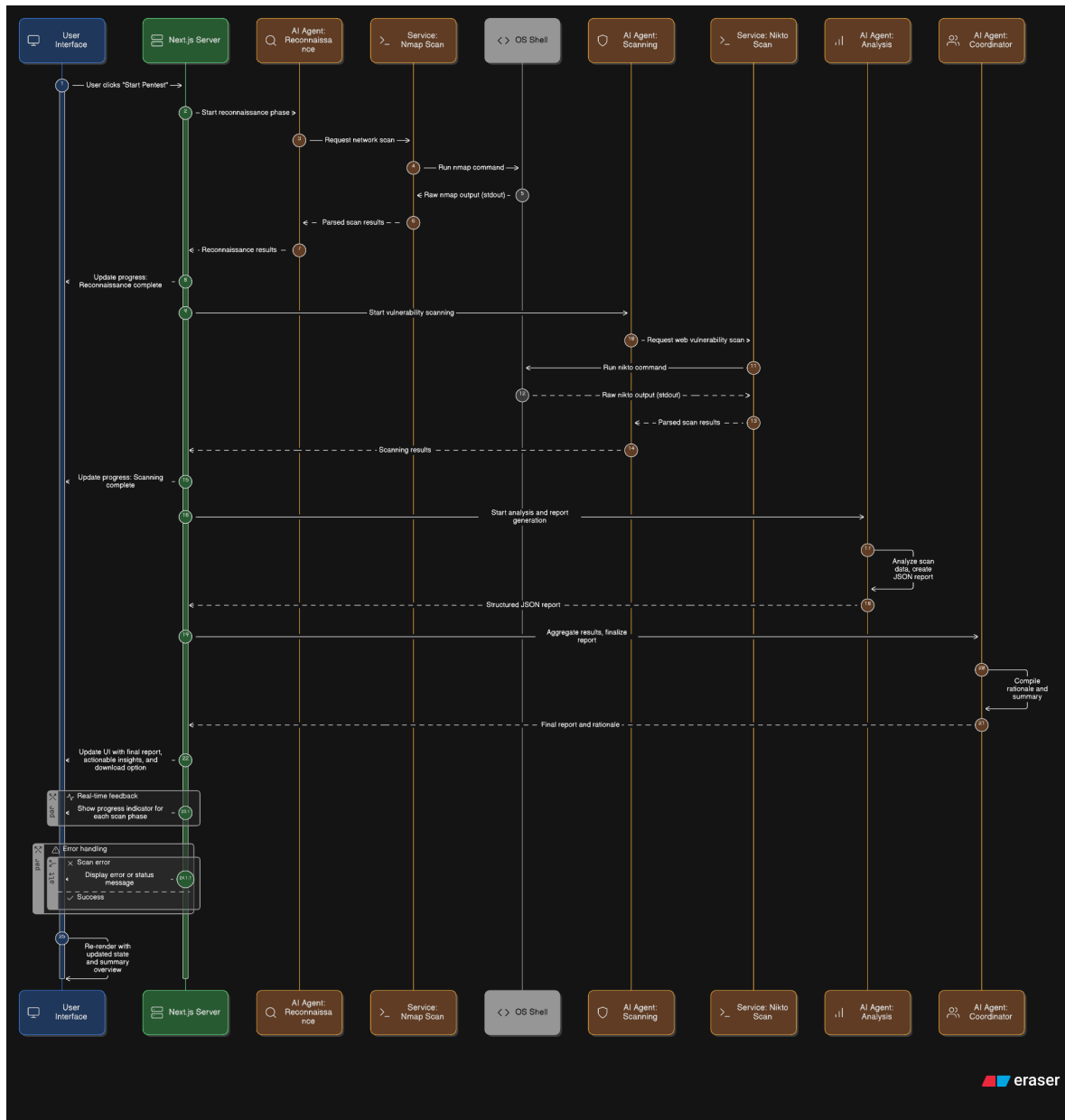


Figure 1: Multi-Agent Interaction Sequence Diagram.

This diagram shows the chronological flow of messages between the application's components. It highlights the handoff of data and control between specialized agents (Reconnaissance, Scanning, Analysis, Coordinator) and their corresponding services.

2.2. Frontend

The frontend provides a user-friendly interface for configuring tests, initiating scans, visualizing the AI's workflow in real-time, and reviewing the final vulnerability report.

- **Framework:** Next.js with React (App Router), leveraging Server Components for optimized performance.
- **UI Components:** ShadCN UI, providing a professional and consistent design system.
- **Styling:** Tailwind CSS for a utility-first styling approach.
- **State Management:** Standard React hooks (useState, useEffect) manage the UI state, including real-time agent logs, discovered findings, and session history.

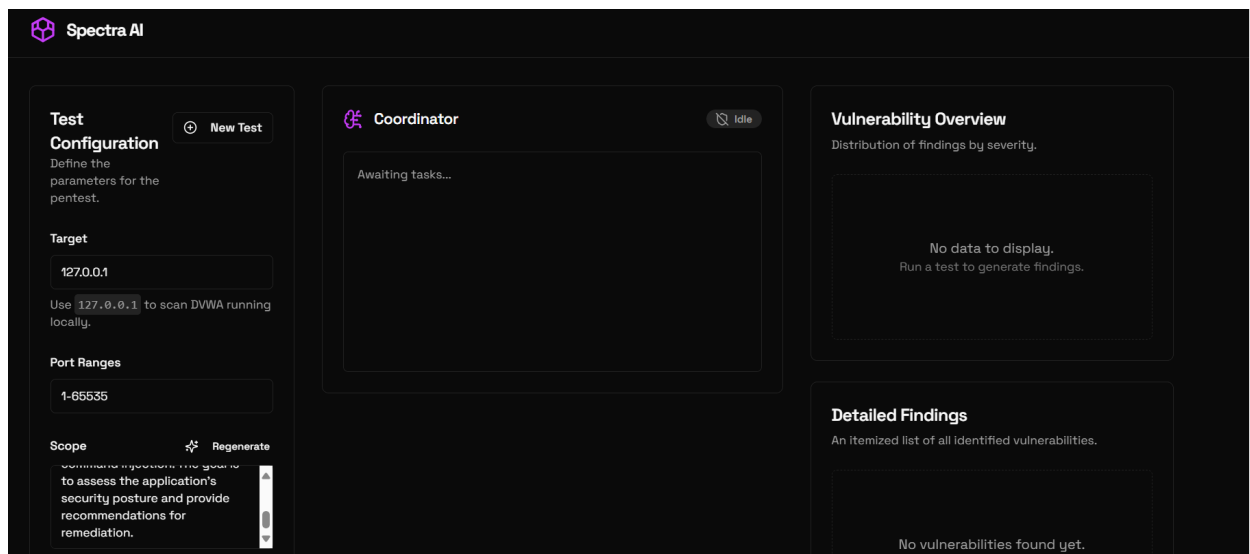


Figure 2: The Main Dashboard.

The user configures the test parameters (Target, Port Ranges, Scope) on the left. The central panel displays real-time logs from the AI agents, and the right panel provides a summary of discovered vulnerabilities.

2.3. Backend (AI Orchestration Engine)

The backend logic is responsible for receiving requests from the frontend, executing the AI-driven penetration testing workflow, invoking command-line tools, and returning structured data to the client.

- **Framework:** The backend logic coexists with the Next.js server in a single **Node.js** process.
- **AI Core: Genkit**, Google's generative AI framework, is the heart of the application. It is used to define, orchestrate, and execute all AI-related tasks and agent flows.

2.4. The Execution Layer: `child_process`

The bridge between the AI's decisions and real-world actions is Node.js's built-in `child_process` module.

The application does not simulate scans; it executes real commands.

1. An AI agent determines that a specific tool (e.g., Nmap) is required.
2. The agent calls a corresponding service function (e.g., `performNmapScan`).
3. This service function utilizes `child_process.exec()` to spawn a new shell process on the host operating system.
4. The command (e.g., `nmap -sV 127.0.0.1`) is executed in the new shell.
5. The server captures the `stdout` and `stderr` from the completed process.
6. This raw text output is returned to the requesting AI agent for analysis.

3. The AI Agent Framework

Spectra AI employs a team of specialized AI agents, each with a distinct role, implemented as "Flows" in Genkit. These agents communicate and pass data sequentially to complete the penetration test.

3.1. Agent: Scope Generator

- **Role:** Test Architect
- **Purpose:** To automate the creation of a standard, professionally worded scope statement based on the target's characteristics, saving the user time and establishing formal parameters.
- **Trigger:** Runs automatically when the target is changed or manually when the user clicks "Regenerate".

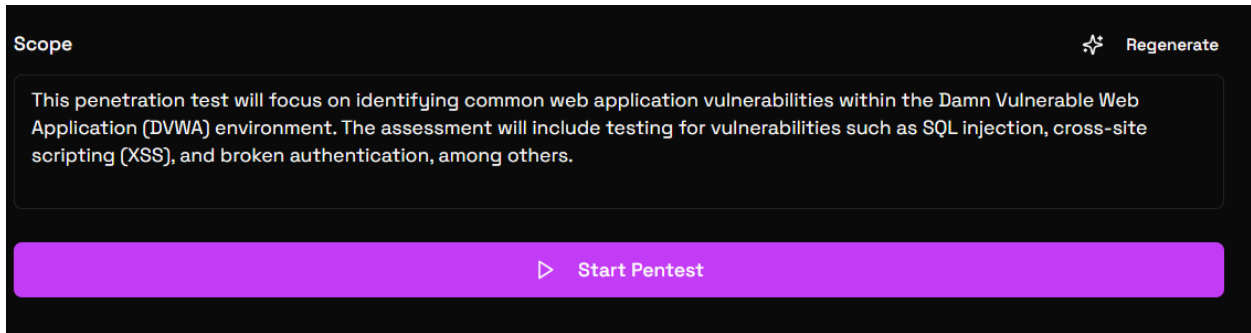


Figure 1a: Test Configuration and Scope

Displays the user-defined scope before initiating the test. The objective is clearly set to assess a Damn Vulnerable Web Application (DVWA) environment for common vulnerabilities. This configuration provides the initial context and goals for the AI agents.

3.2. Agent: Reconnaissance

- **Role:** Scout
- **Purpose:** To perform initial intelligence gathering to determine the target's footprint on the network. A pentester never attacks blindly; this agent mimics that initial information-gathering phase.
- **Logic:** It analyzes the target details and uses Genkit Tools to select and execute the optimal reconnaissance tool (e.g., nmap). It then provides a human-readable summary and structured data for the next agent.

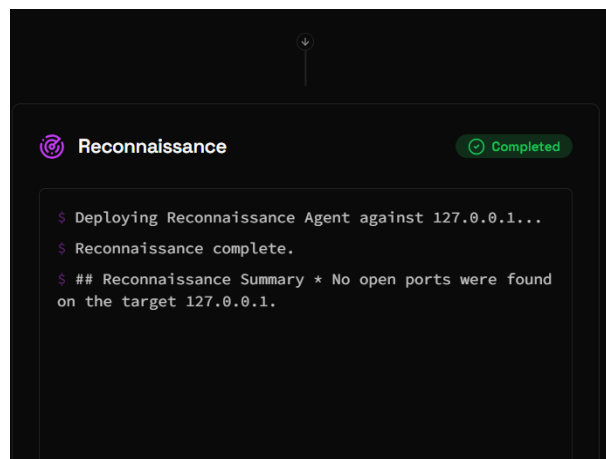


Figure 1b: Reconnaissance Phase Results

Displays the user-defined scope before initiating the test. The objective is clearly set to assess a Damn Vulnerable Web Application (DVWA) environment for common vulnerabilities. This configuration provides the initial context and goals for the AI agents.

3.3. Agent: Scanning

- **Role:** Vulnerability Prober
- **Purpose:** To actively scan for known vulnerabilities based on the intelligence gathered by the Reconnaissance agent. If the "Scout" finds an open door (like a web server), this agent checks if that door is unlocked.
- **Logic:** It analyzes the recon data to identify web services (http, https) on any port. If a web service is detected, it launches a specialized web scanner like nikto to probe for common weaknesses.

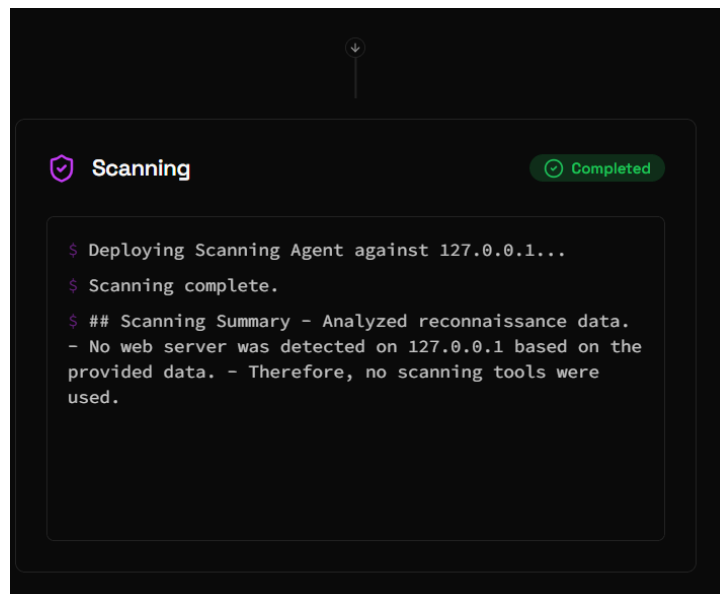


Figure 1c: Scanning Agent's Logical Decision

This log demonstrates the AI's adaptive decision-making. Based on the reconnaissance data, the Scanning agent correctly inferred that no web server was detected. As a result, it logically concluded that running web-specific scanning tools would be pointless, thereby skipping the action.

3.4. Agent: Analysis

- **Role:** Intelligence Analyst
- **Purpose:** To transform raw, noisy text output from tools into a professional, actionable report. This agent acts like a human analyst, reading the raw text and structuring it into a standardized format with titles, descriptions, severity ratings, and remediation advice.
- **Implementation:** This agent uses Genkit's outputSchema with **Zod** to enforce the LLM's output into a clean JSON array (VulnerabilityFindingSchema[]). This is critical for reliably displaying results in the UI.

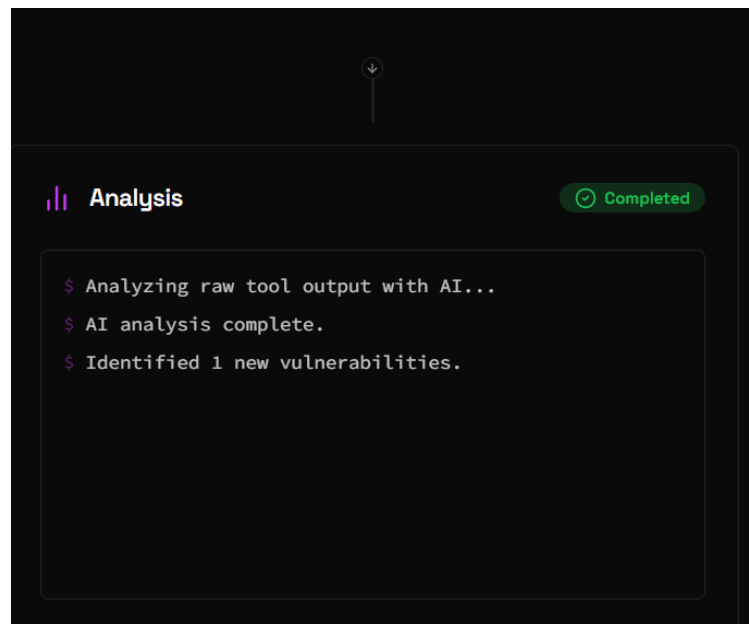


Figure 1d: Analysis Phase Completion

This panel confirms the completion of the Analysis phase. The agent's task is to parse the output from the previous stages. In this scenario, it processed the reports from the Reconnaissance and Scanning agents to prepare for the final summary.

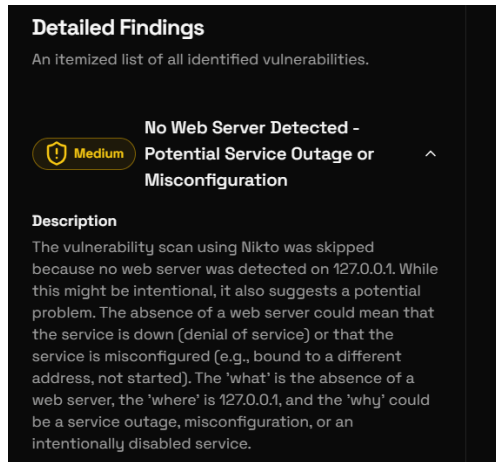


Figure 1e: A Structured Vulnerability Finding.

An example of the structured output generated by the Analysis agent. Raw tool data is converted into clear, actionable items with severity, impact, and recommendations

3.5. Agent: Coordinator

- **Role:** Mission Commander
- **Purpose:** To provide strategic oversight and ensure the test remains ethical and within the defined scope.
- **Logic:** This agent reviews the findings from all other agents to formulate a final rationale and decide on the next steps. It serves as a critical safety and ethics check, verifying that all actions are within scope and acting with caution if the target appears to be unauthorized infrastructure.

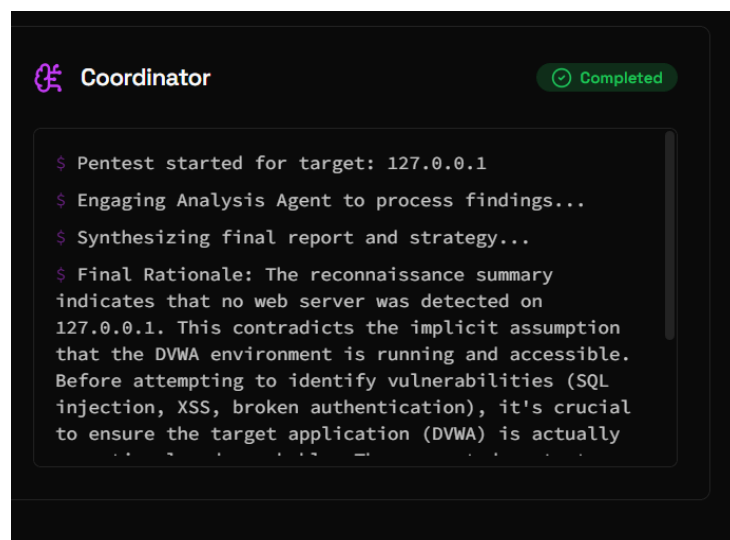


Figure 1e: Coordinator's Final Rationale

This log contains the final, high-level summary from the Coordinator agent. It synthesizes the findings from all previous steps to provide a coherent **Final Rationale**. The AI correctly identifies the core issue: the target was unreachable, which contradicts the initial assumption that a web server was running and accessible.

4. Data Flow & State Management

The following steps detail the journey of data from a user's click to the final report.

1. **Configuration:** The user defines the target and scope in the PentestConfigForm component.
2. **Initiation:** Clicking "Start Pentest" in page.tsx triggers the client-side runPentest function.
3. **Authorization:** A server-side check (isTargetAuthorized) verifies the target is not on a blocklist of prohibited domains (e.g., google.com, amazon.com).
4. **Agent Orchestration:** The runPentest function begins calling the AI agents in sequence. For each agent, it updates the React state to "running," logs the action to the UI, calls the server-side agent, awaits the result, and then updates the state to "completed." The UI re-renders at each step, showing progress in real-time.
5. **Data Passing:** The output of one agent serves as the input for the next. For instance, the raw reconData string from the Reconnaissance agent is passed directly to the Scanning agent.
6. **Final Report:** The Analysis agent receives the raw scanOutput, processes it, and returns a structured findings array. This array is set in the React state, which triggers the "Vulnerability Overview" and "Detailed Findings" UI components to populate with data.

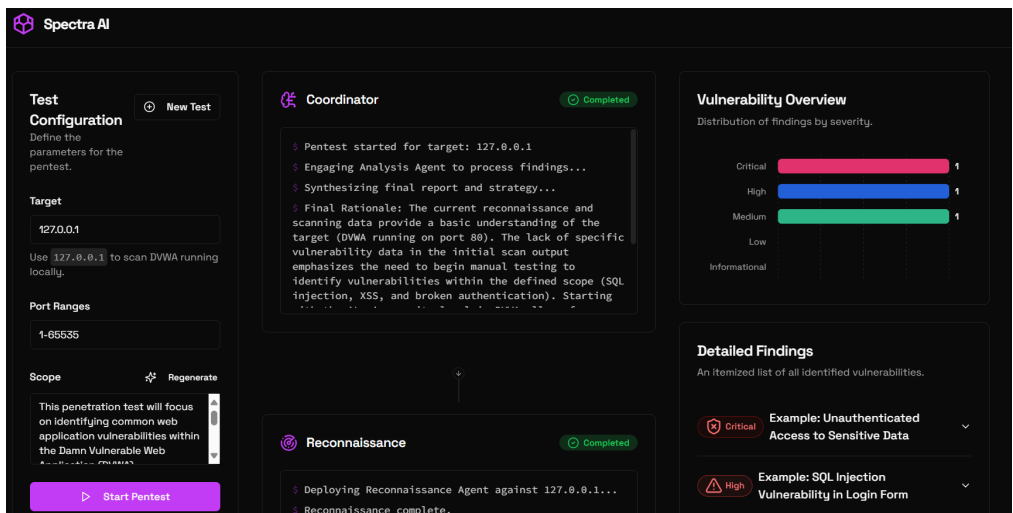


Figure 3: Completed Test Dashboard.

The UI reflects the completed status of all agents. The "Vulnerability Overview" chart is populated, and the "Detailed Findings" section lists the identified issues for user review.

7. Session History: Upon completion, the entire session (configuration, logs, findings) is saved as a single object to the browser's localStorage. This allows the user to review past test results.

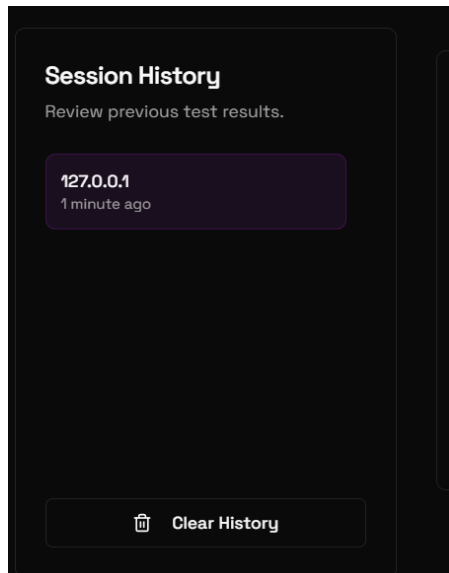


Figure 4: Session History.

Previous test results are stored locally and can be accessed for later review, providing a persistent record of security assessments.

5. Future Enhancements & Extensibility

Spectra AI's modular design allows for significant future enhancements. The framework is built to be easily extended by integrating new cybersecurity tools or developing more specialized AI agents, expanding its testing capabilities and creating more sophisticated workflows.

Integrating New Tools

To broaden the application's assessment capabilities, new command-line tools (e.g., Gobuster for directory brute-forcing, SQLMap for automated SQL injection) can be seamlessly integrated into the AI's toolkit.

1. **Create a Service Function:** In `src/services/scanning.ts`, write a new function that executes the tool's command-line interface using `child_process`.
 2. **Define the Tool for the AI:** In the relevant agent flow, use `ai.defineTool` to describe the new tool to the AI, including its purpose and its input/output schemas (defined with Zod).
 3. **Update the Agent's Prompt:** Modify the agent's prompt to instruct it on the strategic use of the new tool, ensuring it knows when and how to deploy it effectively.
-

Developing New AI Agents

More advanced testing logic can be implemented by creating new AI agents with specialized roles, such as an "Exploitation Agent" to test vulnerabilities or a "Reporting Agent" to generate different report formats.

1. **Create the Agent Flow:** In a new file under `src/ai/flows/`, define the agent's core logic, its primary prompt, and its data schemas with Zod.
2. **Register the Agent:** Import and expose the new agent flow in `src/ai/dev.ts` so the system recognizes it.
3. **Integrate into Workflow:** In the main `runPentest` function in `src/app/page.tsx`, call the new agent at the appropriate point in the testing sequence.
4. **Update UI and Types:** Add the agent's name to the `AgentName` type in `src/lib/types.ts` and create a corresponding status card in the UI to display its progress.

6. Conclusion

Spectra AI demonstrates the powerful potential of combining generative AI with deterministic security tools to automate complex workflows. By architecting a system where specialized AI agents collaborate—from reconnaissance to analysis—the application successfully mimics the cognitive processes of a human penetration tester while maintaining the speed and scalability of automated software.

This project highlights a mastery of full-stack development using **Next.js** and **Node.js**, as well as advanced AI engineering skills utilizing Google's **Genkit** framework. The robust "human-in-the-loop" design, enforced through strict **Zod** schemas and the **Coordinator Agent**, ensures that the system remains safe, ethical, and accurate.

Ultimately, Spectra AI is not just a concept but a functional prototype that bridges the gap between natural language intent and real-world system execution, serving as a scalable foundation for the next generation of automated security assessment tools.